Bitácora Lab S09 · Mensajería conRabbitMQ

Claudia Alin Rojas Jiménez

1 · El problema (Parte 0)

Con `notificacion` apagado hice una transferencia de 50 de Ana a Beto. El dinero sí se movió (Ana quedó en 950 y Beto en 550), pero la API respondió 500 y la transferencia se quedó en `ABONADA`. Esta prueba se hizo con la rama base, antes de tocar el código.

```
docker compose stop notificacion
curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/jso
n' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}'
curl -s 'http://localhost:8080/api/transferencias?estado=ABONADA'
curl -s http://localhost:8080/api/cuentas/002180000000000001
curl -s http://localhost:8080/api/cuentas/002180000000000002
docker compose start notificacion
```

```
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 01:54:05 GMT

{"timestamp":"2026-10-19T01:54:05.783+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
```

Figura 1. Con notificación apagado, la transferencia de 50 responde 500 Internal Server Error.

```
$ curl -s 'http://localhost:8080/api/transferencias?estado=ABONADA'
[{"id":1,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50.00,"fecha":"2026-10-
19T01:54:05.452258Z","estado":"ABONADA","motivo":null,"saldoOrigenTrasCargo":950.00,"intentosDeCompensacion":0,
"actualizada":"2026-10-19T01:54:05.596257Z"}]
```

Figura 2. La consulta por estado ABONADA muestra la transferencia que se quedó a medias.

```
$ curl -s http://localhost:8080/api/cuentas/002180000000000001
{"id":1,"clabe":"002180000000000001","titular":"Ana","saldo":950.00}
$ curl -s http://localhost:8080/api/cuentas/002180000000000002
{"id":2,"clabe":"002180000000000002","titular":"Beto","saldo":550.00}
```

Figura 3. Saldos después del error: Ana en 950 y Beto en 550. El dinero sí se movió.

Por qué pasó: la saga ya había hecho el cargo, el abono y los asientos, pero al final se quedó esperando la respuesta HTTP de `notificacion`. Como nadie contestó, esa llamada lanzó una excepción antes de `marcarCompletada()` y el error le llegó al cliente, aunque la operación ya estaba hecha.

2 · Publicar el evento

Ya con los cambios, la transferencia respondió 201 y quedó en `COMPLETADA` aunque `notificacion` estaba apagado. Como todavía no había ninguna cola enlazada al exchange, el broker regresó el evento (por `mandatory`) y el log lo marcó como no publicado, así que ese aviso se perdió.

```
docker compose up --build -d transferencia
docker compose stop notificacion
curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/jso
n' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}'
docker compose logs transferencia | grep evento
```

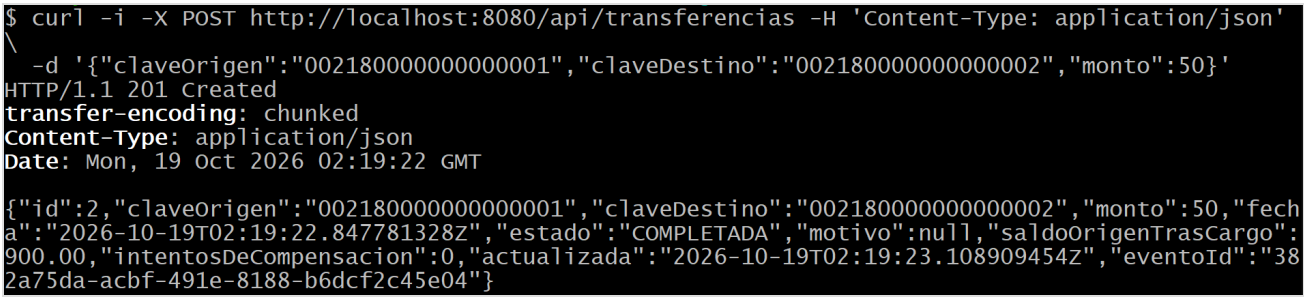


Figura 4. Con notificación apagado, la transferencia ya responde 201, queda en COMPLETADA y trae su eventoId.

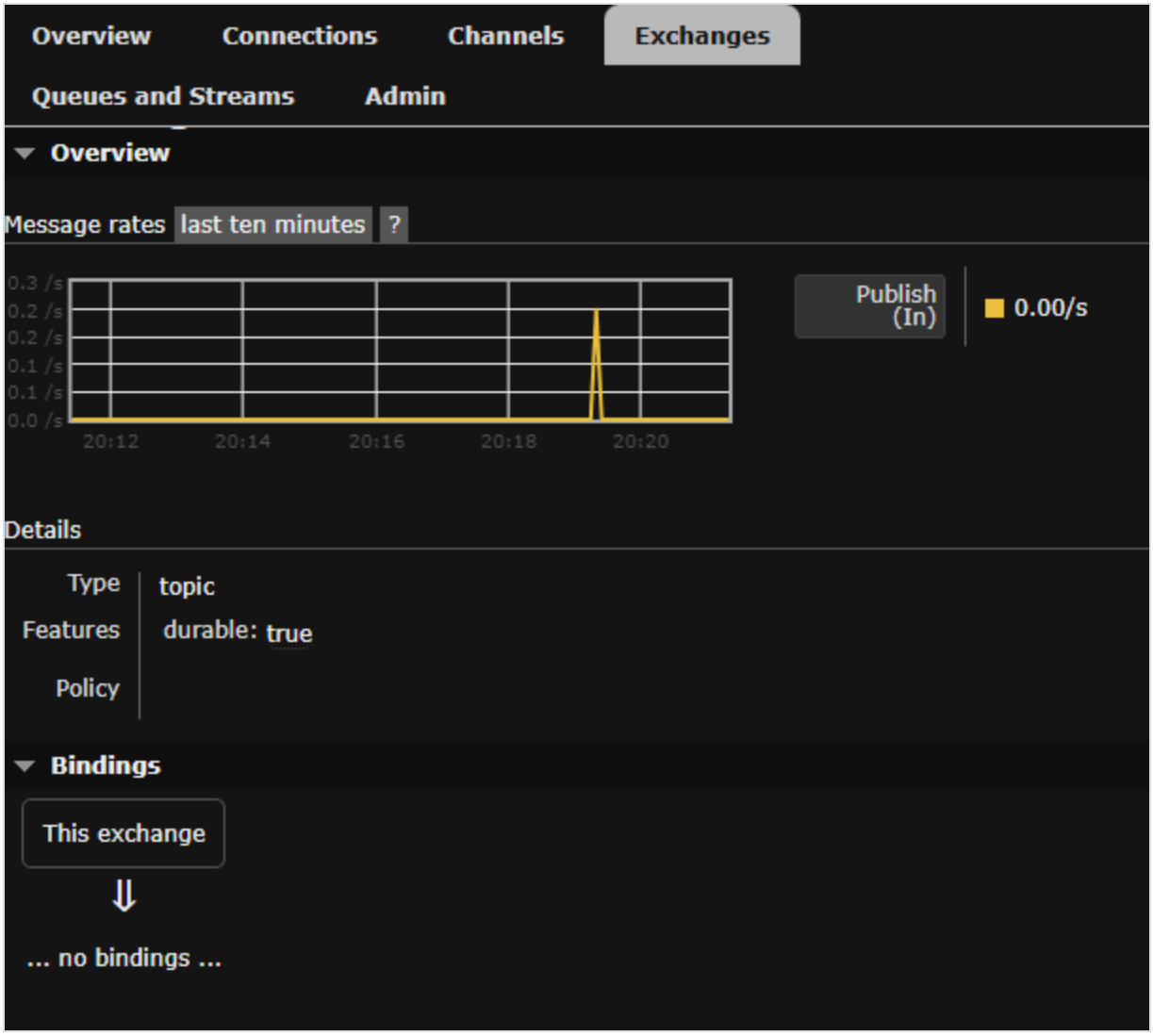


Figura 5. El exchange transferencias en el panel: tipo topic, durable y todavía sin colas enlazadas.

```
$ docker compose logs transferencia | grep -i evento
transferencia-1  | 2026-10-19T02:19:23.127Z ERROR 1 --- [transferencia] [nio-8080-exec-3] m.m.t
ransferencia.TransferenciaService    : Transferencia 2: quedo terminada pero el evento Transfere
nciaRealizada 382a75da-acbf-491e-8188-b6dcf2c45e04 NO se publico: Ninguna cola recibio el event
o 382a75da-acbf-491e-8188-b6dcf2c45e04 con la routing key transferencia.realizada (NO_ROUTE)
```

Figura 6. Log de transferencia: ninguna cola recibió el evento, así que no se publicó.

El código clave

Pieza	Qué hace
`Transferencia.eventoId`	UUID que nace con la fila y se guarda con ella; si el evento se publica dos veces, lleva el mismo id.
`EventoTransferencia`	Record propio del contrato, separado de la entidad: un cambio de columna no rompe a los consumidores.
`ConfiguracionMensajeria`	Declara el exchange `transferencias` (topic, durable) y el convertidor JSON. Transferencia no declara colas.

Pieza	Qué hace
`PublicadorDeEventos`	Publica persistente y espera el ack del broker; lanza excepción si no hay conexión, si llega nack, si no hay respuesta o si ninguna cola lo recibió (`mandatory`).
`TransferenciaService`	Ya no llama a `NotificacionCliente`. Guarda `COMPLETADA` y luego publica; `avisar()` atrapa la falla y solo la registra en el log.

Qué garantiza el publisher confirm

Lo que garantiza: que el broker recibió el mensaje y se hizo cargo de él. Como el mensaje va persistente y la cola es durable, ya quedó en disco; y con `mandatory`, además, que por lo menos una cola lo recibió.

Lo que no garantiza: que alguien lo procese. Si `notificacion` está apagado, el ack llega igual, porque el mensaje solo se queda esperando en la cola. Tampoco cubre lo que pasa antes de publicar: si el proceso muere entre `guardar(t)` y `avisar(...)`, la base ya dice `COMPLETADA` y el evento nunca se mandó. Es el problema de la doble escritura.

3 · Desacople en tiempo

Tres transferencias (11, 12 y 13) respondieron 201 con `notificacion` apagado; sus eventos esperaron en la cola y al encenderlo llegaron los seis avisos.

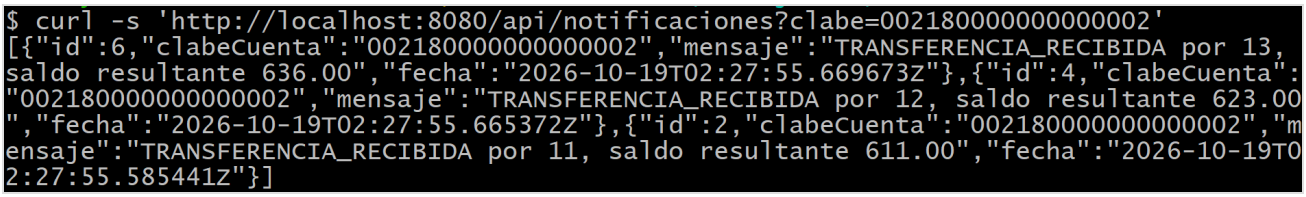
```
docker compose up --build -d notificacion    # al arrancar declara la cola notificacion.tra
nsferencias
docker compose stop notificacion
for m in 11 12 13; do
  curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8080/api/transferencias
\
    -H 'Content-Type: application/json' \
    -d "{\"claveOrigen\":\"00218000000000001\",\"claveDestino\":\"00218000000000002\",\"m
onto\":$m}"
done
docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowl
edged
docker compose start notificacion
curl -s 'http://localhost:8080/api/notificaciones?clabe=00218000000000002'
```

```
$ for m in 11 12 13; do
  curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8080/api/transferencias \
    -H 'Content-Type: application/json' \
    -d "{\"claveOrigen\":\"002180000000000001\",\"claveDestino\":\"002180000000000002\",\"monto
\":$m}"
done
201
201
201
```

Figura 7. Tres transferencias (11, 12 y 13) con notificación apagado; las tres responden 201.

```
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_un
acknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     3       0
```

Figura 8. La cola notificacion.transferencias con 3 mensajes Ready esperando a su consumidor.



```
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":6,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13,
saldo resultante 636.00","fecha":"2026-10-19T02:27:55.669673Z"},{"id":4,"clabeCuenta":
"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00
","fecha":"2026-10-19T02:27:55.665372Z"},{"id":2,"clabeCuenta":"002180000000000002","m
ensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00","fecha":"2026-10-19T0
2:27:55.585441Z"}]
```

Figura 9. Al encender notificación, la bandeja de Beto recibe los avisos por 11, 12 y 13.

La cola es durable y el consumidor confirma con `basicAck` solo después de guardar los avisos. El productor y el consumidor ya no tienen que estar vivos al mismo tiempo.

La transferencia revertida

```
FALLAR_ABONO_A_CLABE=002180000000000002 docker compose up -d transferencia
curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/jso
n' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":150}'
curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000001'
docker compose up -d transferencia
```

```
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: ap
plication/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","
monto":150}'
HTTP/1.1 502 Bad Gateway
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Mon, 19 Oct 2026 02:31:35 GMT

{"type":"about:blank","title":"Transferencia no realizada","status":502,"detail
":"No se pudo abonar a la CLABE 002180000000000002, asi que se devolvieron 150
a la CLABE 002180000000000001: su saldo quedo como estaba.","instance":"/transf
erencias","transferencia":6,"estado":"COMPENSADA","motivo":"El abono a la CLABE
 002180000000000002 fallo: Falla provocada con FALLAR_ABONO_A_CLABE=00218000000
0000002"}
```

```
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000001'
[{"id":7,"clabeCuenta":"002180000000000001","mensaje":"DEVOLUCION_TRANSFERENCIA
 por 150, saldo resultante 864.00","fecha":"2026-10-19T02:31:36.558360Z"},{"id"
:5,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 13,
saldo resultante 864.00","fecha":"2026-10-19T02:27:55.669615Z"},{"id":3,"clabeC
uenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 12, saldo resu
ltante 877.00","fecha":"2026-10-19T02:27:55.665320Z"},{"id":1,"clabeCuenta":"00
2180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 11, saldo resultante 889
.00","fecha":"2026-10-19T02:27:55.585353Z"}]
```

Figura 10. Con la falla provocada, la transferencia de 150 queda COMPENSADA (502) y Ana recibe el aviso DEVOLUCION_TRANSFERENCIA.

4 · El duplicado

Matar `notificacion` entre el commit y el ack hizo que el mismo evento se entregara dos veces y dejara avisos repetidos; con la marca por `eventoId`, la segunda entrega guarda 0 avisos.

Antes del arreglo

```
# con la pausa antes del ack y todavía sin idempotencia
PAUSA_ANTES_DEL_ACK_MS=15000 docker compose up --build -d notificacion
# transferencia de 77 y, durante los 15 s de pausa:
docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowl
edged
docker compose kill notificacion
docker compose up -d notificacion            # sin la variable: pausa 0
docker compose logs notificacion | grep consumidor
```

```
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     0       1
```

Figura 11. Durante la pausa antes del ack, el mensaje ya se entregó pero sigue Unacked.

```
$ docker compose logs notificacion | grep consumidor
notificacion-1  | 2026-10-19T02:37:06.083Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTra
nsferencias       : [consumidor] TransferenciaRealizada de la transferencia 7 (evento 04632480-ec3d-44a5-ae8a-b
38cc1c27178, redelivered=true): 2 aviso(s) nuevo(s)
```

Figura 12. Tras matar notificación, el mensaje se vuelve a entregar con redelivered=true y guarda 2 avisos más.

```
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":11,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 77, saldo resultante 713.00"
,"fecha":"2026-10-19T02:37:06.001480Z"},{"id":9,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_REC
IBIDA por 77, saldo resultante 713.00","fecha":"2026-10-19T02:36:29.920962Z"},{"id":6,"clabeCuenta":"0021800000
00000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-19T02:27:55.66967
3Z"},{"id":4,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.
00","fecha":"2026-10-19T02:27:55.665372Z"},{"id":2,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_
RECIBIDA por 11, saldo resultante 611.00","fecha":"2026-10-19T02:27:55.585441Z"}]
```

Figura 13. La bandeja de Beto con el aviso por 77 repetido.

El arreglo

- Tabla `evento_procesado` con `evento_id` como llave primaria.
- `marcarSiEsNuevo` es una sola sentencia: `INSERT … ON CONFLICT (evento_id) DO NOTHING`, que regresa 1 si es nuevo y 0 si ya estaba.
- Se llama al principio de `avisarDeTransferencia`, dentro del mismo `@Transactional` que `saveAll(avisos)`. Si regresa 0, no se guarda nada y el consumidor hace ack igual.
- No es "consultar y luego insertar" (dos entregas podrían colarse entre las dos llamadas) ni "atrapar la violación de llave" (en Postgres deja la transacción abortada).
- Si algo falla después de la marca, el rollback se lleva marca y avisos juntos y la siguiente entrega empieza desde cero.

Después del arreglo

```
# ya con la idempotencia, mismo experimento con una transferencia de 88
PAUSA_ANTES_DEL_ACK_MS=15000 docker compose up --build -d notificacion
docker compose kill notificacion             # durante la pausa
docker compose up -d notificacion
docker compose logs notificacion | grep -E "consumidor|ya se habia procesado"
```

```
$ docker compose logs notificacion | grep -E "consumido
r|ya se habia procesado"
notificacion-1  | 2026-10-19T02:43:09.209Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.Notifi
cacionService     : [notificacion] el evento 22bcfa0c-6ea2-4fde-8d8e-019f1d7883f4 ya se habia procesado: no se
repiten los avisos
notificacion-1  | 2026-10-19T02:43:09.212Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTra
nsferencias       : [consumidor] TransferenciaRealizada de la transferencia 8 (evento 22bcfa0c-6ea2-4fde-8d8e-0
19f1d7883f4, redelivered=true): 0 aviso(s) nuevo(s)
```

Figura 14. Con la idempotencia, la segunda entrega del evento de 88 guarda 0 avisos nuevos.

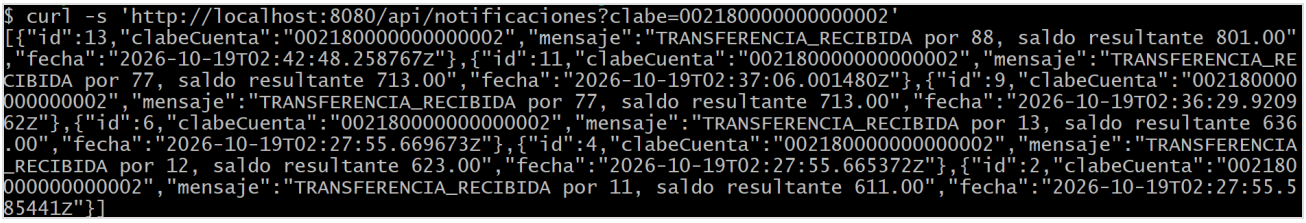


Figura 15. La bandeja tiene un solo aviso por 88.

5 · La DLQ

Sin DLQ, el mensaje `{"hola":` se queda trabado en Unacked; con la DLQ, el consumidor lo rechaza sin reencolar y el broker lo mueve a `notificacion.transferencias.dlq`.

Antes del arreglo

Desde el panel publiqué en el exchange `transferencias`, con routing key `transferencia.realizada`, un mensaje que no es JSON válido: `{"hola":`. Mientras tanto dejé corriendo `docker compose logs -f notificacion`.

El `readValue` lanzó una excepción de Jackson. En el log primero aparece `Execution of Rabbit message listener failed` (Spring la envuelve en `ListenerExecutionFailedException`) y más abajo, en el `Caused by`, la causa real: `JsonEOFException: Unexpected end-of-input`. Como el ack es manual y el método nunca llegó al `basicAck`, nadie respondió por ese mensaje y se quedó en **Unacked**, ocupando uno de los 5 lugares del `prefetch`. Una transferencia normal de 99 sí generó su aviso, pero con 5 mensajes así el consumidor se quedaría sin lugares y dejaría de procesar todo.

```
notificacion-1  | 2026-10-19T02:48:34.631Z  WARN 1 --- [notificacion] [ntContainer#0-1] s.a.r.l.ConditionalReje
ctingErrorHandler : Execution of Rabbit message listener failed.
notificacion-1  |
notificacion-1  | org.springframework.amqp.rabbit.support.ListenerExecutionFailedException: Listener method 'public void mx.mexibanco.n
otificacion.mensajeria.ConsumidorDeTransferencias.recibir(org.springframework.amqp.core.Message,com.rabbitmq.client.Channel) throws jav
a.io.IOException' threw exception
notificacion-1  |       at org.springframework.amqp.rabbit.listener.adapter.MessagingMessageListenerAdapter.invokeHandler(MessagingMess
ageListenerAdapter.java:287) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.adapter.MessagingMessageListenerAdapter.invokeHandlerAndProcessResu
lt(MessagingMessageListenerAdapter.java:225) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.adapter.MessagingMessageListenerAdapter.onMessage(MessagingMessageL
istenerAdapter.java:149) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.doInvokeListener(AbstractMessageLi
stenerContainer.java:1682) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.actualInvokeListener(AbstractMessa
geListenerContainer.java:1604) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.invokeListener(AbstractMessageList
enerContainer.java:1592) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.doExecuteListener(AbstractMessageL
istenerContainer.java:1583) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.executeListenerAndHandleException(
AbstractMessageListenerContainer.java:1528) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.lambda$executeListener$8(AbstractM
essageListenerContainer.java:1506) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at io.micrometer.observation.Observation.observe(Observation.java:499) ~[micrometer-observation-1.13.4.jar!/:1.
13.4]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.executeListener(AbstractMessageLis
tenerContainer.java:1506) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.SimpleMessageListenerContainer.doReceiveAndExecute(SimpleMessageLis
tenerContainer.java:1082) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.SimpleMessageListenerContainer.receiveAndEx
ecute(SimpleMessageListenerContainer.java:1018) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at com.fasterxml.jackson.core.base.ParserMinimalBase._reportInvalidEOF(ParserMinimalBase.java:585) ~[jackson-co
re-2.17.2.jar!/:2.17.2]
notificacion-1  |       at com.fasterxml.jackson.core.json.UTF8StreamJsonParser._skipColon2(UTF8StreamJsonParser.java:3226) ~[jackson-c
ore-2.17.2.jar!/:2.17.2]
notificacion-1  |       at com.fasterxml.jackson.core.json.UTF8StreamJsonParser._skipColon(UTF8StreamJsonParser.java:3141) ~[jackson-co
re-2.17.2.jar!/:2.17.2]
notificacion-1  |       at com.fasterxml.jackson.core.json.UTF8StreamJsonParser.nextToken(UTF8StreamJsonParser.java:761) ~[jackson-core
```

```
$ docker compose logs notificacion | grep -i -A2 "caused by"
notificacion-1  | Caused by: com.fasterxml.jackson.core.io.JsonEOFException: Unexpected end-of-input within/between Object entries
notificacion-1  |  at [Source: REDACTED (`StreamReadFeature.INCLUDE_SOURCE_IN_LOCATION` disabled); line: 1, column: 9]
notificacion-1  |       at com.fasterxml.jackson.core.base.ParserMinimalBase._reportInvalidEOF(ParserMinimalBase.java:585) ~[jackson-co
re-2.17.2.jar!/:2.17.2]
```

Figura 16. El mensaje mal formado hace fallar al listener; en el Caused by aparece la JsonEOFException de Jackson.

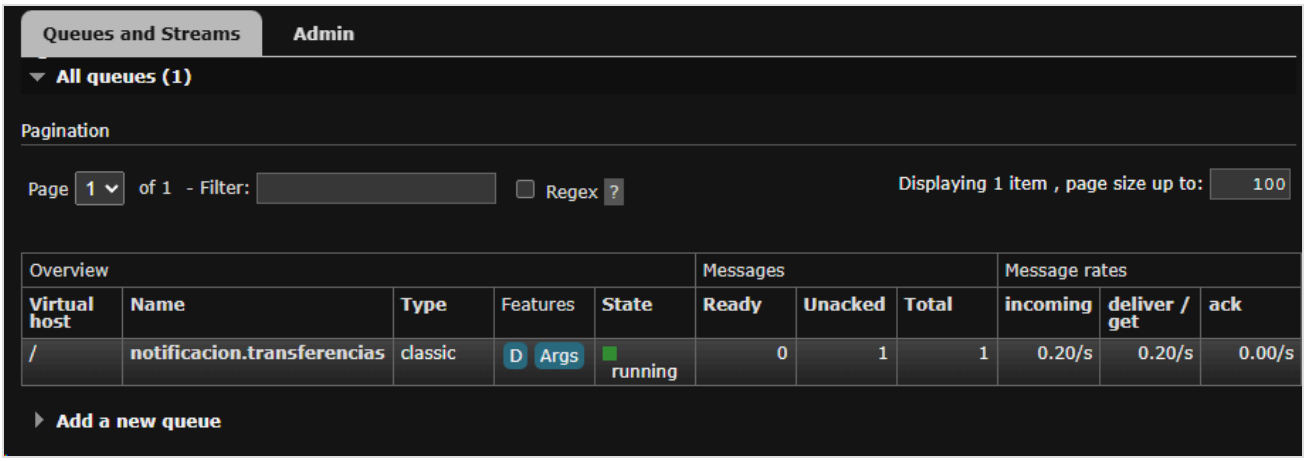


Figura 17. El mensaje envenenado se queda atorado en la cola: 0 Ready y 1 Unacked.

```
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":99}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 02:59:16 GMT

{"id":9,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":99,"fecha":"2026-10-19T0
2:59:15.981475851Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":600.00,"intentosDeCompensacion":
0,"actualizada":"2026-10-19T02:59:16.066615129Z","eventoId":"8df1c7c8-ea1e-4b94-91f5-b00ecc5f1ebe"}
```

```
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":15,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 99, saldo resultante 900.00"
,"fecha":"2026-10-19T02:59:16.077246Z"},{"id":13,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RE
CIBIDA por 88, saldo resultante 801.00","fecha":"2026-10-19T02:42:48.258767Z"},{"id":11,"clabeCuenta":"00218000
0000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 77, saldo resultante 713.00","fecha":"2026-10-19T02:37:06.001
480Z"},{"id":9,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 77, saldo resultante 71
3.00","fecha":"2026-10-19T02:36:29.920962Z"},{"id":6,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCI
A_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-19T02:27:55.669673Z"},{"id":4,"clabeCuenta":"00218
0000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2026-10-19T02:27:55.
665372Z"},{"id":2,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante
 611.00","fecha":"2026-10-19T02:27:55.585441Z"}]
```

Figura 18. Aun con el mensaje atorado, una transferencia normal de 99 genera su aviso.

Después del arreglo

```
docker compose stop notificacion
# panel → Queues and Streams → notificacion.transferencias → Delete (la cola vieja no tiene
argumentos de DLX)
docker compose up --build -d notificacion
# se vuelve a publicar {"hola": desde el panel
docker compose logs notificacion | grep "va a la DLQ"
```

- La cola principal lleva `x-dead-letter-exchange = transferencias.dlx` y `x-dead-letter-routing-key = notificacion.transferencias`.
- `transferencias.dlx` (direct, durable) enlaza `notificacion.transferencias.dlq` (durable) con esa clave.
- JSON ilegible, o un JSON que no cumple el contrato (sin `eventoId`, `tipo` desconocido, sin saldos): `basicNack(etiqueta, false, false)`.
- Si falla nuestra base, el mensaje está bien: `basicNack(etiqueta, false, true)` tras 1 s, para reintentar.

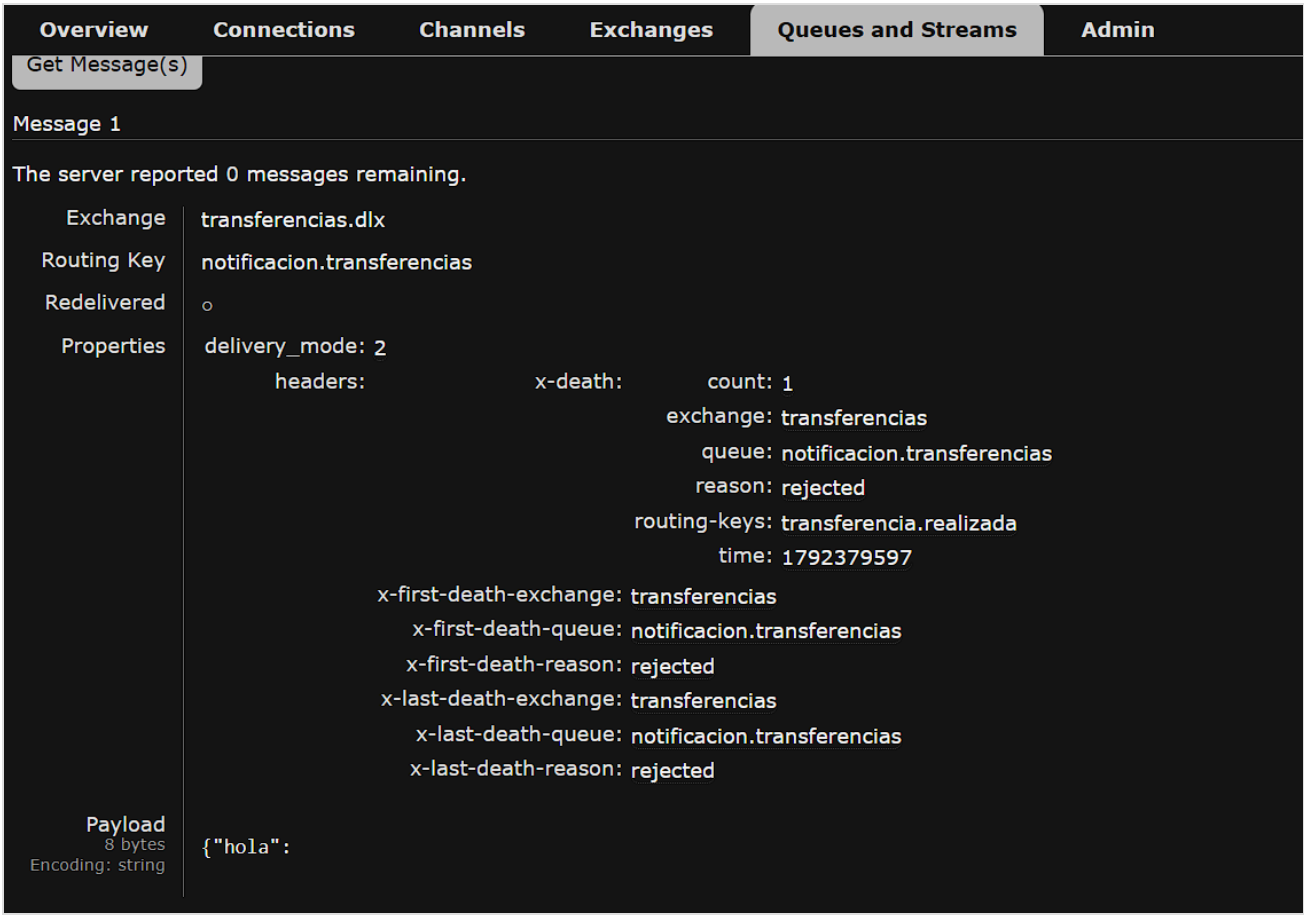


Figura 19. Con la DLQ, el mensaje termina en notificacion.transferencias.dlq con la cabecera x-death y reason: rejected.

```
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":66}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 03:15:43 GMT

{"id":10,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":66,"fecha":"2026-10-19T
03:15:43.030487465Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":534.00,"intentosDeCompensacion"
:0,"actualizada":"2026-10-19T03:15:43.102560415Z","eventoId":"337428b7-b5bd-48b9-9084-ed41a060e64d"}
```

```
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=00218000000000002'
[{"id":17,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 66, saldo resultante 966.00"
,"fecha":"2026-10-19T03:15:43.170228Z"},{"id":15,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RE
CIBIDA por 99, saldo resultante 900.00","fecha":"2026-10-19T02:59:16.077246Z"},{"id":13,"clabeCuenta":"00218000
0000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 88, saldo resultante 801.00","fecha":"2026-10-19T02:42:48.258
767Z"},{"id":11,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 77, saldo resultante 7
13.00","fecha":"2026-10-19T02:37:06.001480Z"},{"id":9,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENC
IA_RECIBIDA por 77, saldo resultante 713.00","fecha":"2026-10-19T02:36:29.920962Z"},{"id":6,"clabeCuenta":"0021
8000000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-19T02:27:55
.669673Z"},{"id":4,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultant
e 623.00","fecha":"2026-10-19T02:27:55.665372Z"},{"id":2,"clabeCuenta":"00218000000000002","mensaje":"TRANSFER
ENCIA_RECIBIDA por 11, saldo resultante 611.00","fecha":"2026-10-19T02:27:55.585441Z"}]
```

Figura 20. Después del mensaje malo, una transferencia de 66 responde 201 y su aviso llega a la bandeja.

6 · Preguntas de cierre

1. Si RabbitMQ está caído cuando la saga llega a `COMPLETADA`, ¿qué pasa con el aviso?

En la versión sin outbox, la transferencia responde 201 y queda `COMPLETADA`, pero `publicar()` no logra conectarse (se rinde a los 2 s por el `connection-timeout`) y `avisar()` solo deja en el log que el evento no se publicó. Ese evento no queda guardado en ningún lado, así que cuando RabbitMQ regresa nadie lo vuelve a mandar y el aviso se pierde; es lo que pasó con la transferencia 13 en la sección 7. El publisher confirm no lo arregla porque solo dice si el envío salió bien o mal: detecta la falla, pero no guarda el evento ni lo reintenta.

2. ¿Por qué el ack va después de guardar y no antes?

Si el ack fuera antes y `notificacion` muriera entre el ack y el commit, el broker ya habría borrado el mensaje sin que los avisos se guardaran: el aviso se pierde (entrega como máximo una vez). Con el ack después, si muere entre el commit y el ack, el broker vuelve a entregar el mensaje con `redelivered=true` y el aviso se repite (entrega al menos una vez). Es mejor repetir que perder, y con la idempotencia por `eventoId` la segunda entrega ya no hace nada, como se ve en la sección 4.

3. Elige y di qué sacrificaste.

- **Elección:** avisar con un evento en RabbitMQ en vez de una llamada HTTP que espera respuesta.
- **Porque:** una transferencia que ya movió el dinero no debería fallar por culpa de un aviso, que puede esperar.
- **Sacrifico:** saber en el momento que el aviso salió. El 201 ya no lo garantiza, el aviso puede llegar tarde o repetido, y ahora hay un broker más que mantener y depurar.

7 · Reto: Transactional Outbox

Con RabbitMQ apagado la transferencia se completa, el evento espera en la tabla `outbox` de transferencia y sale solo cuando el broker vuelve.

```
# ya con el outbox
docker compose up --build -d transferencia
docker compose stop rabbitmq
curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/jso
n' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":2}'
docker compose logs transferencia | grep -i outbox     # "sigue pendiente"
docker compose ps transferencia                         # sigue healthy
docker compose start rabbitmq
# espera unos segundos
docker compose logs transferencia | grep "publicado desde el outbox"
curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
```

Antes del outbox (7.1)

Con RabbitMQ apagado y el código de la sección 6, la transferencia 13 (monto 1) respondió 201 `COMPLETADA`, pero el log dice `NO se publico ... Connect timed out`. Al volver el broker, su aviso nunca llegó: en la bandeja, el saldo más alto de Beto es 968, y el aviso de la transferencia 13 habría dicho 969. Los dos avisos "por 1" que sí aparecen son de intentos anteriores, hechos con el broker arriba. La prueba final está en el 7.2: el aviso por 2 deja a Beto en 971 (968 + 1 + 2), así que el peso de la transferencia 13 sí llegó a su cuenta, pero su aviso se perdió.

```
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json'   -d '{"claveOri
gen":"002180000000000001","claveDestino":"002180000000000002","monto":1}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 03:21:49 GMT

{"id":13,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1,"fecha":"2026-10-19T0
3:21:45.128541919Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":531.00,"intentosDeCompensacion":
0,"actualizada":"2026-10-19T03:21:45.188241269Z","eventoId":"0d1027b4-4ba4-44fe-8208-676418ae06ee"}
```

```
$ docker compose logs transferencia | grep -i "NO se publico"
transferencia-1  | 2026-10-19T03:21:49.146Z ERROR 1 --- [transferencia] [nio-8080-exec-2] m.m.transferencia.Tra
nsferenciaService   : Transferencia 13: quedo terminada pero el evento TransferenciaRealizada 0d1027b4-4ba4-44f
e-8208-676418ae06ee NO se publico: No se pudo mandar el evento 0d1027b4-4ba4-44fe-8208-676418ae06ee al broker:
java.net.SocketTimeoutException: Connect timed out
```

```
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":21,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 968.00",
"fecha":"2026-10-19T03:21:11.236774Z"},{"id":19,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_REC
IBIDA por 1, saldo resultante 967.00","fecha":"2026-10-19T03:20:43.959206Z"},{"id":17,"clabeCuenta":"0021800000
00000002","mensaje":"TRANSFERENCIA_RECIBIDA por 66, saldo resultante 966.00","fecha":"2026-10-19T03:15:43.17022
8Z"},{"id":15,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 99, saldo resultante 900
.00","fecha":"2026-10-19T02:59:16.077246Z"},{"id":13,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCI
A_RECIBIDA por 88, saldo resultante 801.00","fecha":"2026-10-19T02:42:48.258767Z"},{"id":11,"clabeCuenta":"0021
80000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 77, saldo resultante 713.00","fecha":"2026-10-19T02:37:06
.001480Z"},{"id":9,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 77, saldo resultant
e 713.00","fecha":"2026-10-19T02:36:29.920962Z"},{"id":6,"clabeCuenta":"002180000000000002","mensaje":"TRANSFER
ENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-19T02:27:55.669673Z"},{"id":4,"clabeCuenta":"0
02180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2026-10-19T02:27
:55.665372Z"},{"id":2,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo result
ante 611.00","fecha":"2026-10-19T02:27:55.585441Z"}]
```

Figura 21. Sin outbox y con RabbitMQ apagado, la transferencia 13 responde 201, el log dice que el evento no se publicó y su aviso nunca llega.

Con el outbox (7.2)

```
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":2}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 03:29:05 GMT

{"id":14,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":2,"fecha":"2026-10-19T0
3:29:05.631640712Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":529.00,"intentosDeCompensacion":
0,"actualizada":"2026-10-19T03:29:05.788536879Z","eventoId":"9fdfb594-6117-483d-a6dd-6aa3746afab1"}
```

Figura 22. Con el outbox y RabbitMQ apagado, la transferencia de 2 responde 201 en COMPLETADA.

```
$ docker compose logs transferencia | grep -i "outbox"     # varias líneas "sigue pendiente"
transferencia-1  | 2026-10-19T03:28:38.037Z  WARN 1 --- [transferencia] [           main] o.h.engine.jdbc.spi.SqlExceptionHelper   : constraint "uktrqfcw89nuj9ywxa16cniydm0" of relation "outbox" does not exist, skipping
transferencia-1  | 2026-10-19T03:29:05.796Z  INFO 1 --- [transferencia] [nio-8080-exec-6] m.m.mensajeria.outbox.BandejaDeSalida    : Transferencia 14: COMPLETADA (evento TransferenciaRealizada 9fdfb594-6117-483d-a6dd-6aa3746afab1 en el outbox)
transferencia-1  | 2026-10-19T03:29:08.718Z  WARN 1 --- [transferencia] [   scheduling-1] m.m.mensajeria.outbox.RelayDeEventos     : Outbox: el evento TransferenciaRealizada 9fdfb594-6117-483d-a6dd-6aa3746afab1 de la transferencia 14 sigue pendiente (intento 1): No se pudo mandar el evento 9fdfb594-6117-483d-a6dd-6aa3746afab1 al broker: java.net.UnknownHostException: rabbitmq: Name does not resolve
transferencia-1  | 2026-10-19T03:29:09.734Z  WARN 1 --- [transferencia] [   scheduling-1] m.m.mensajeria.outbox.RelayDeEventos     : Outbox: el evento TransferenciaRealizada 9fdfb594-6117-483d-a6dd-6aa3746afab1 de la transferencia 14 sigue pendiente (intento 2): No se pudo mandar el evento 9fdfb594-6117-483d-a6dd-6aa3746afab1 al broker: java.net.UnknownHostException: rabbitmq
transferencia-1  | 2026-10-19T03:29:10.739Z  WARN 1 --- [transferencia] [   scheduling-1] m.m.mensajeria.outbox.RelayDeEventos     : Outbox: el evento TransferenciaRealizada 9fdfb594-6117-483d-a6dd-6aa3746afab1 de la transferencia 14 sigue pendiente (intento 3): No se pudo mandar el evento 9fdfb594-6117-483d-a6dd-6aa3746afab1 al broker: java.net.UnknownHostException: rabbitmq
transferencia-1  | 2026-10-19T03:29:11.755Z  WARN 1 --- [transferencia] [   scheduling-1] m.m.mensajeria.outbox.RelayDeEventos     : Outbox: el evento TransferenciaRealizada 9fdfb594-6117-483d-a6dd-6aa3746afab1 de la transferencia 14 sigue pendiente (intento 4): No se pudo mandar el evento 9fdfb594-6117-483d-a6dd-6aa3746afab1 al broker: java.net.UnknownHostException: rabbitmq
transferencia-1  | 2026-10-19T03:29:12.769Z  WARN 1 --- [transferencia] [   scheduling-1] m.m.mensajeria.outbox.RelayDeEventos     : Outbox: el evento TransferenciaRealizada 9fdfb594-6117-483d-a6dd-6aa3746afab1 de la transferencia 14 sigue pendiente (intento 5): No se pudo mandar el evento 9fdfb594-6117-483d-a6dd-6aa3746afab1 al broker: java.net.UnknownHostException: rabbitmq
```

```
$ docker compose ps transferencia                          # sigue healthy
NAME                              IMAGE                             COMMAND                  SERVICE         CREATED          STATUS                    PORTS
mexi-banco-v06-transferencia-1   mexi-banco-v06-transferencia   "java -jar app.jar"   transferencia   About a minute ago   Up About a minute (healthy)   8080/tcp
```

Figura 23. Mientras el broker está caído, el relay reintenta cada segundo y transferencia sigue healthy.

```
$ docker compose logs transferencia | grep "publicado desde el outbox"
transferencia-1  | 2026-10-19T03:29:57.633Z  INFO 1 --- [transferencia] [   scheduling-1] m.m.mensajeria.outbox.RelayDeEventos     : Transferencia 14: evento TransferenciaRealizada publicado desde el outbox (9fdfb594-6117-483d-a6dd-6aa3746afab1, intento 41)
```

```
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":23,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 2, saldo resultante 971.00","fecha":"2026-10-19T03:29:57.637104Z"},{"id":21,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 968.00","fecha":"2026-10-19T03:21:11.236774Z"},{"id":19,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 967.00","fecha":"2026-10-19T03:20:43.959206Z"},{"id":17,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 66, saldo resultante 966.00","fecha":"2026-10-19T03:15:43.170228Z"},{"id":15,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 99, saldo resultante 900.00","fecha":"2026-10-19T02:59:16.077246Z"},{"id":13,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 88, saldo resultante 801.00","fecha":"2026-10-19T02:42:48.258767Z"},{"id":11,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 77, saldo resultante 713.00","fecha":"2026-10-19T02:37:06.001480Z"},{"id":9,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 77, saldo resultante 713.00","fecha":"2026-10-19T02:36:29.920962Z"},{"id":6,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-19T02:27:55.669673Z"},{"id":4,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2026-10-19T02:27:55.665372Z"},{"id":2,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00","fecha":"2026-10-19T02:27:55.585441Z"}]
```

Figura 24. Al volver RabbitMQ, el evento sale desde el outbox y el aviso por 2 llega a la bandeja de Beto.

- `BandejaDeSalida.guardarConEvento` guarda el estado final y la fila de `outbox` en **una** transacción local. Es un bean aparte porque `@Transactional` no aplica en métodos privados ni en llamadas dentro de la misma clase.
- `RelayDeEventos`, cada 1 s, toma las filas pendientes con `SELECT … FOR UPDATE SKIP LOCKED`, las publica con `PublicadorDeEventos` (que espera la confirmación) y las marca. Si el broker no confirma, la fila sigue pendiente y se corta la vuelta.
- **Si el relay publica y muere antes de marcar la fila**, el evento sale otra vez con el mismo `eventoId`; lo salva el consumidor idempotente de la sección 4.
- **Health:** RabbitMQ entraba en `/actuator/health`, así que con el broker apagado transferencia saldría `unhealthy` aunque funcione. Se quitó con `management.health.rabbit.enabled: false`; el atraso ahora se ve en el log del relay y en las filas con `publicado` nulo.